# NESTED LOOP
# CALENDAR GENERATOR

### Python Project Report

## Project Overview

This project generates a formatted monthly calendar by using the supplied month, year, starting weekday, and number of days. Nested loops align the dates under the correct weekdays. The program also checks month names, month lengths, leap years, and input ranges.

## Weekday numbering

```
0 = Sunday   1 = Monday   2 = Tuesday   3 = Wednesday
4 = Thursday 5 = Friday   6 = Saturday
```

Sample run: March 2026, starting weekday = 0 (Sunday), number of days = 31.

## 1. Problem Statement

Develop a Python program that generates a monthly calendar using nested loops. It must place dates under the correct weekday, accept a supplied starting weekday and number of days, and reject invalid months or month lengths.

## 2. Objectives

• Generate a month calendar in a clear seven-column format.

• Use nested loops to print weeks and weekdays.

• Align the first date according to the supplied starting weekday.

• Validate month name, year, starting weekday, and number of days.

• Handle leap years when checking February.

## 3. Assumptions and Rules

• Weekday number 0 represents Sunday and 6 represents Saturday.

• Month names are entered in English; upper- and lower-case are accepted.

• Year must be 1 or greater.

• January, March, May, July, August, October, and December have 31 days; April, June, September, and November have 30 days.

• February has 28 days, or 29 in a leap year.

• The supplied number of days must match the selected month and year.

## 4. Algorithm

1. Start the program.

2. Read month, year, starting weekday, and number of days.

3. Find the valid number of days for that month and year.

4. Check the month, year, starting weekday, and supplied days.

5. Print the weekday headings from Sunday to Saturday.

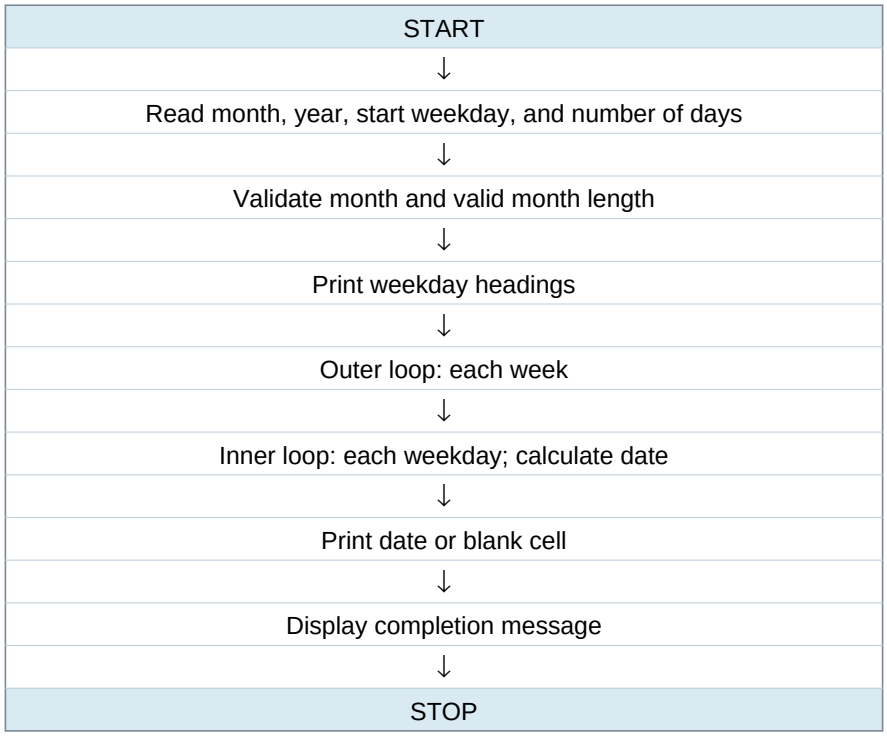
6. Use an outer loop for each week and an inner loop for each weekday.

7. Calculate the date position using week number, weekday position, and start offset.

8. Print the date when it is within the month; otherwise print a blank cell.

9. Stop after the final date and display a success message.

## 5. Flowchart




## 6. Pseudocode

```
BEGIN
    INPUT month, year, start_weekday, number_of_days
    valid_days = days_in_selected_month(month, year)
    IF month or inputs are invalid:
        DISPLAY error and STOP
    PRINT weekday headings
    FOR each week:
        FOR each weekday:
            day = week * 7 + weekday - start_weekday + 1
            IF 1 <= day <= number_of_days:
                PRINT day
            ELSE:
                PRINT blank space
        PRINT new line
    DISPLAY success message
END
```

## 7. Python Source Code

Save this file as **nested_loop_calendar_generator.py**. Run it in VS Code and enter the month, year, starting weekday, and number of days when prompted.

```
# Nested Loop Calendar Generator

def is_leap_year(year):
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)


def expected_days(month, year):
    month_days = {
        "january": 31, "february": 29 if is_leap_year(year) else 28,
        "march": 31, "april": 30, "may": 31, "june": 30,
        "july": 31, "august": 31, "september": 30,
        "october": 31, "november": 30, "december": 31
    }
    return month_days.get(month.lower())


def print_calendar(month, year, start_weekday, number_of_days):
    day_names = ["SUN", "MON", "TUE", "WED", "THU", "FRI", "SAT"]
    valid_days = expected_days(month, year)

    if valid_days is None:
        print("Error: Invalid month name.")
        return
    if year < 1:
        print("Error: Year must be 1 or greater.")
        return
    if start_weekday < 0 or start_weekday > 6:
        print("Error: Starting weekday must be 0 to 6 (Sunday to Saturday).")
        return
    if number_of_days != valid_days:
        print(f"Error: {month} {year} must have {valid_days} days.")
        return

    print("=" * 35)
    print(f"          {month.upper()} {year}")
    print("=" * 35)
    for day_name in day_names:
        print(f"{day_name:>4}", end="")
    print()
    print("-" * 35)

    # Nested loops: outer loop creates each week; inner loop prints each weekday.
    for week in range(6):
```

## 7. Python Source Code (continued)

```
        for weekday in range(7):
            day = week * 7 + weekday - start_weekday + 1
            if 1 <= day <= number_of_days:
                print(f"{day:4}", end="")
            else:
                print("    ", end="")
        print()
        if (week + 1) * 7 - start_weekday >= number_of_days:
            break

    print("-" * 35)
    print("Calendar generated successfully.")


def main():
    print("NESTED LOOP CALENDAR GENERATOR")
    print("Weekday number: 0=Sunday, 1=Monday, ... 6=Saturday")
    month = input("Enter month name: ").strip()
    try:
        year = int(input("Enter year: "))
        start_weekday = int(input("Enter starting weekday number (0-6): "))
        number_of_days = int(input("Enter number of days in the month: "))
    except ValueError:
        print("Error: Year, weekday, and number of days must be integers.")
        return

    print_calendar(month, year, start_weekday, number_of_days)

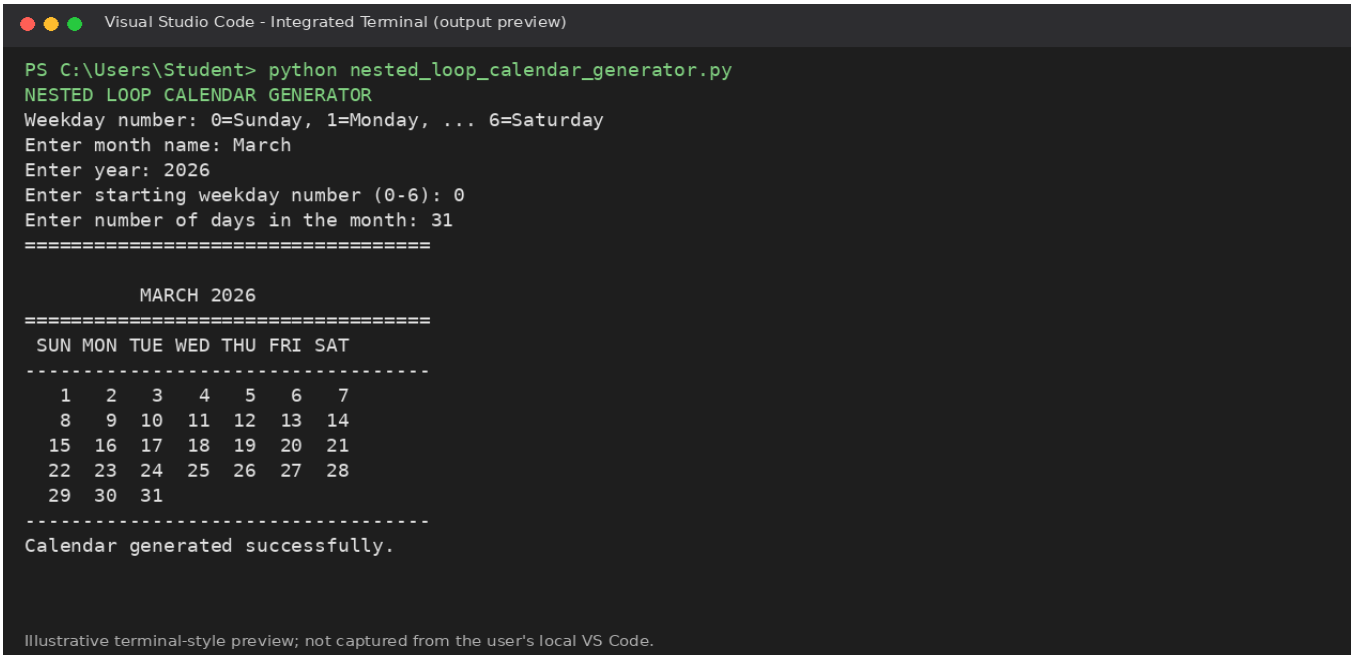

if __name__ == "__main__":
    main()
```

## 8. Test Cases

| Test case | Input / condition | Expected result |
|---|---|---|
| Sample month | March 2026, start 0, days 31 | Calendar prints with March 1 under Sunday |
| Leap year | February 2024, days 29 | Accepted |
| Non-leap year | February 2025, days 29 | Rejected; February must have 28 days |
| Invalid weekday | Starting weekday 7 | Rejected; must be 0 to 6 |
| Wrong month length | April 2026, days 31 | Rejected; April must have 30 days |
| Invalid month | Month name: Smarch | Rejected with an error message |

## 9. Output Screenshot

The image below is an illustrative terminal-style output preview. It is not a direct screenshot from the user's local VS Code application.




To capture a genuine VS Code screenshot, open the Python file, select Terminal → New Terminal, run **python nested_loop_calendar_generator.py**, enter March, 2026, 0, and 31, then take a screenshot of the editor and terminal.

## 10. Conclusion

The program uses nested loops to generate a neatly aligned monthly calendar. It validates supplied values, handles leap-year February correctly, and reports invalid month lengths and weekday values.